

rebound_rust

Provenance of the Pure-Rust Ports of REBOUND 5.1.1 and REBOUNDx 5.1.0

rustSolveIt / Windows 11 port verification record

August 26, 2026

Contents

REBOUND and REBOUNDx in Pure Rust — Complete Guide and Provenance	4
Part I — Understanding	5
1. What are these programs?	5
REBOUND	5
REBOUNDx	5
2. What is a "port", and why do one?	5
3. What "bit-for-bit identical" means	6
Part II — Using it	7
4. What you need installed	7
4a. Rust	7
4b. Microsoft Visual Studio Build Tools	7
4c. (Only if you want to rebuild the C originals) GnuWin32 Make	7
5. Five-minute quick start	7
6. Cookbook: how to do the common things	9
Choose an integrator	9
Set the timestep (fixed-step integrators)	10
Add a particle by position and velocity	10
Add a particle by orbital elements	10
Read orbital elements back out	10
Integrate	10
Check accuracy with energy	10
Do something during the run (heartbeat)	11
Turn on collisions	11
Save and reload a simulation	11
Watch it in a browser	11
Set options that live on the integrator	11

7. Adding extra physics with REBOUNDx	12
Example: general-relativistic precession	12
Where parameters live	12
Example: tides and spin	12
Reading a parameter back	13
The available effects	13
8. Every build command, in one place	13
Part III — Provenance: how it was made and checked	15
9. The machine and the tools	15
GLFW (for completeness)	15
10. Building the C originals (the reference)	15
10a. REBOUND	15
10b. REBOUNDx	16
11. How the translation was done	17
The one structural problem, and its solution	17
12. File-by-file accounting: REBOUND	17
13. File-by-file accounting: REBOUNDx	18
Core machinery	18
Forces	19
Operators, steppers and REBOUNDx's own integrators	19
14. Deviations from the C, and why each is safe	20
15. The complete verification record	21
The record at a glance	21
15.0 The foundation: which maths functions agree?	22
15.1 The integrator matrix — 63 configurations	22
15.2 The shearing sheet — the acceptance test	24
15.3 Orbital derivatives — 65 functions	24
15.4 Frequency analysis	25
15.5 Simulationarchive — cross-language round trips	25
15.6 The web server	26
15.7 add_fmt and the built-in datasets	26
15.8 REBOUNDx: the tides_spin acceptance tests	26
15.9 REBOUNDx binary files: C writes, Rust reads, and the reverse	27
15.10 Three real defects the new test suite found	28
16. The one known difference: pow	30
17. Lint policy: why we waive some compiler suggestions	31
The one waiver you must never "clean up"	31
18. Known limitations	32

19. How to reproduce every result yourself	32
Step 5 — build the C comparison harnesses	33
Step 6 — run each matched pair and compare	34
20. Credit, citation and licensing	34
The originals	34
How to cite	35
A note on the upstream projects' AI policy	35
Versions translated	35

REBOUND and REBOUNDx in Pure Rust — Complete Guide and Provenance

What this document is. Everything about two computer programs we translated: what they are, how to install and use them, exactly how they were built, and the complete evidence that the translation is correct.

Who it is for. Anyone — including someone who has never written a line of code. Technical terms are explained the first time they appear. Every command you need is printed in full, so you never have to open another document.

The one-sentence summary. We rewrote two professional astronomy programs from the C language into the Rust language, and proved the rewrite produces *exactly* the same numbers — every bit of every number — as the originals.

Part I — Understanding

1. What are these programs?

REBOUND

REBOUND answers the question: *given some objects in space, where will they be later?*

You tell it the objects — their masses, positions and velocities — and it calculates how gravity moves them. The objects can be planets orbiting a star, moons orbiting a planet, asteroids, or billions of ice chunks in Saturn's rings. It is used in hundreds of published astronomy papers.

It was written in the **C** programming language by **Hanno Rein** (University of Toronto) and collaborators. It is free and open-source.

The hard part of such a program is *accuracy over long times*. If you simulate the Solar System for a billion years, tiny errors at each step pile up. **REBOUND** contains about a dozen different **integrators** — different mathematical recipes for stepping forward in time — each making a different trade between speed and accuracy. Choosing the right one is most of the skill in using it.

REBOUNDx

REBOUNDx ("REBOUND eXtras") adds physics beyond plain gravity. Written by **Dan Tamayo**, Hanno Rein and collaborators, it lets you switch on effects like:

- **general relativity** — Einstein's correction to Newton's gravity, which makes Mercury's orbit slowly rotate;
- **tides** — the stretching a planet feels from its star, which slowly circularises orbits (and is why our Moon always shows us the same face);
- **spin evolution** — how a planet's rotation axis tips and drifts over time;
- **radiation pressure** — sunlight physically pushing on dust grains;
- **migration** — planets spiralling inward or outward through a gas disk.

You pick the effects you want and attach them to a **REBOUND** simulation.

2. What is a "port", and why do one?

A **port** is a rewrite of a program in a different language that keeps exactly what it does.

We ported both programs from **C** to **Rust**.

Why? C is fast, and **REBOUND**'s C is well-written — but the C language itself allows a category of mistakes that are easy to make and hard to find:

- reading or writing past the end of a list (**buffer overrun**),
- using memory after it has been given back (**use-after-free**),
- forgetting to give memory back at all (**memory leak**).

These do not always crash. Sometimes they silently corrupt a number, and you publish a wrong result.

Rust refuses to compile code that could do these things. That guarantee is enforced by the compiler, not by the programmer's care. Our translation goes further and switches on `#![forbid(unsafe_code)]`, which removes even Rust's own escape hatch. So there is no place in this code where those bugs can hide.

We also used **zero third-party libraries**. The only thing our code depends on is Rust's own standard library. Nothing is downloaded when you build it.

The catch. A rewrite that changes the numbers is useless — worse than useless, because you might trust it. So the whole project rests on proving the numbers did not change. That proof is Part III of this document.

3. What "bit-for-bit identical" means

Computers store decimal numbers as 64 ones-and-zeros, in a standard format called **IEEE-754 double precision**. Those 64 ones-and-zeros are called **bits**.

When we say two results are **bit-for-bit identical**, we mean *all 64 bits are the same*, for every number. Not "the same to 10 decimal places" — identical.

Why insist on something so strict? Because these simulations are **chaotic**. A difference in the very last bit — about 1 part in 10 million billion — grows each step. After enough steps, two runs that differed by one bit look completely different. Both are still valid answers, but they are not the *same* answer.

So bit-for-bit agreement is the only test that actually proves the arithmetic is the same. Anything weaker ("agrees to 8 digits") could be hiding a real bug that has not yet had time to grow.

To test it, both programs print every number as its raw bits in **hexadecimal** (base 16, digits 0-9 and a-f). The number 1.0 prints as `3ff0000000000000`. We then compare the files character by character.

ULP. You will see "ULP" below. It means *Unit in the Last Place*: the smallest difference two neighbouring representable numbers can have. A "1 ULP difference" is the smallest possible disagreement.

Part II — Using it

4. What you need installed

You need a Windows PC (64-bit Intel or AMD) and two free downloads.

One thing to know before you read any command in this document Every command below says **C:\work**. That is a stand-in for *whatever folder you put this project in* — it is not a real folder on your machine and not one you have to create with that exact name. So if you cloned everything into C:\Users\Sam\Documents\astronomy, then wherever this document says “ `cd C:\work\rebound_rust` ” you type “ `cd C:\Users\Sam\Documents\astronomy\rebound_` ” Only the front part changes. Everything after C:\work\ - the folder names rebound_rust, reboundx_rust, rebound\rebound\src, porttest and so on - is real and must match exactly, because that is the layout the code expects. If you would rather not retype it every time, tell the terminal once: `'powershell $work = "C:\Users\Sam\Documents\astronomy" cd "$work\rebound_rust" "`

4a. Rust

Go to <<https://rustup.rs>> and download `rustup-init.exe`. Run it and accept the defaults (this installs the MSVC toolchain, which is what we want).

Check it worked — open a terminal and type:

```
rustc --version
```

You should see something like `rustc 1.91.1`.

4b. Microsoft Visual Studio Build Tools

Needed because Rust uses Microsoft’s linker on Windows.

Go to <<https://visualstudio.microsoft.com/downloads/>>, scroll to "Tools for Visual Studio", download **Build Tools for Visual Studio**, run it, and tick "**Desktop development with C++**".

4c. (Only if you want to rebuild the C originals) GnuWin32 Make

```
winget install GnuWin32.Make
```

You do **not** need this just to use the Rust code. It is only for reproducing the verification in Part III.

5. Five-minute quick start

Let us simulate a star with one planet.

Step 1 — make a new project.

```
cd C:\Users\youruser\Desktop
cargo new my_first_simulation
cd my_first_simulation
```

`cargo` is Rust’s build tool. `cargo new` makes a folder with a small starter program inside.

Step 2 — tell it to use our library.

Open the file `Cargo.toml` in that folder and add the last line below:

```
[package]
name = "my_first_simulation"
version = "0.1.0"
edition = "2021"

[dependencies]
rebound_rs = { path = "C:/work/rebound_rust" }
```

(Use forward slashes / even on Windows — that is what this file expects.)

Step 3 — write the simulation.

Replace everything in `src/main.rs` with:

```
use rebound_rs::*;

fn main() {
    // Create an empty simulation.
    let mut sim = reb_simulation_create();
    let r = &mut sim;

    // Choose units where the gravitational constant G is 1.
    // With this choice, one unit of length is one AU (the Earth-Sun
    // distance), masses are in solar masses, and 2*pi units of time is
    // one year.
    r.G = 1.0;

    // Choose the integrator. "ias15" is the most accurate general choice.
    reb_simulation_set_integrator(r, "ias15");

    // Add the star: one solar mass, sitting at the origin, not moving.
    let mut star = reb_particle::default();
    star.m = 1.0;
    reb_simulation_add(r, star);

    // Add a planet using orbital elements instead of position/velocity:
    // m = mass, a = size of orbit, e = how squashed the orbit is
    // e = 0 is a perfect circle; e = 0.9 is a long thin ellipse.
    reb_simulation_add_fmt(r, "m a e", &[
        reb_fmt_arg::d(1e-3), // mass: about one Jupiter
        reb_fmt_arg::d(1.0),   // a: 1 AU
        reb_fmt_arg::d(0.1),   // e: slightly elliptical
    ]);

    // Shift everything so the centre of mass does not drift.
    reb_simulation_move_to_com(r);

    // Record the starting energy so we can check accuracy at the end.
    let energy_start = reb_simulation_energy(r);

    // Integrate forward for 100 years (remember: 2*pi = 1 year).
    reb_simulation_integrate(r, 100.0 * 2.0 * std::f64::consts::PI);

    // Print where the planet ended up.
    let p = r.particles[1];
    println!("after {:.1} years:", r.t / (2.0 * std::f64::consts::PI));
```



```
println!(" planet position: x = {:.6}, y = {:.6}", p.x, p.y);

// Energy should be almost perfectly conserved. This is the standard
// way to check an N-body simulation is behaving.
let energy_end = reb_simulation_energy(r);
let drift = ((energy_end - energy_start).abs());
println!(" relative energy drift: {:.3e} (smaller is better)", drift);
}
```

Step 4 — run it.

```
cargo run --release
```

`-release` turns on optimisation; without it the program runs perhaps 10-50 times slower. Always use `-release` for real work.

You should see something like:

```
after 100.0 years:
planet position: x = 0.246238, y = -0.945196
relative energy drift: 3.331e-16
```

An energy drift of about 10^{-16} is the smallest a computer can represent — the simulation is as accurate as double-precision arithmetic allows.

Congratulations, you have run an N-body simulation.

6. Cookbook: how to do the common things

All snippets assume use `rebound_rs::*`; and a simulation `r`.

Choose an integrator

```
reb_simulation_set_integrator(r, "ias15");
```

Which one should you use?

Name	Use it when	Notes
"ias15"	default choice ; you want accuracy and do not know what else to pick	adaptive: chooses its own step size
"whfast"	long runs of a planetary system, speed matters	you must set <code>r.dt</code> yourself; very fast
"mercurius"	planets that occasionally have close encounters	switches to IAS15 near encounters
"trace"	close encounters, and you need time-reversibility	
"saba"	high-accuracy long-term planetary runs	family of high-order methods
"sei"	shearing-sheet (a patch of a planetary ring)	
"leapfrog"	teaching, simple problems	
"janus"	exactly reversible integer-based arithmetic	
"eos"	embedded operator splitting	
"bs"	you also need to evolve your own equations alongside	adaptive

Name	Use it when	Notes
"none"	you want particles not to move	

Set the timestep (fixed-step integrators)

```
r.dt = 0.01;
```

Rule of thumb: about 1/20th of the shortest orbital period in your simulation.

Add a particle by position and velocity

```
let mut p = reb_particle::default();
p.m = 1e-3;
p.x = 1.0; p.y = 0.0; p.z = 0.0;
p.vx = 0.0; p.vy = 1.0; p.vz = 0.0;
p.r = 4.6e-5; // physical radius (needed for collisions/tides)
reb_simulation_add(r, p);
```

Add a particle by orbital elements

```
reb_simulation_add_fmt(r, "m a e inc Omega omega f", &[
    reb_fmt_arg::d(1e-3), // mass
    reb_fmt_arg::d(5.2), // semi-major axis (orbit size)
    reb_fmt_arg::d(0.048), // eccentricity (0 = circle)
    reb_fmt_arg::d(0.02), // inclination (tilt), radians
    reb_fmt_arg::d(1.75), // longitude of ascending node, radians
    reb_fmt_arg::d(0.257), // argument of pericentre, radians
    reb_fmt_arg::d(0.0), // true anomaly (where on the orbit), radians
]);
```

You may give any subset. Unspecified angles default to 0. There is also a built-in Solar System:

```
reb_simulation_add_fmt(r, "solar system", &[]); // Sun + 8 planets
reb_simulation_add_fmt(r, "outer solar system", &[]); // Sun + Jupiter..Neptune
```

(These require $r.G = 1.0$.)

Read orbital elements back out

```
let orbit = reb_orbit_from_particle(r.G, r.particles[1], r.particles[0]);
println!("a = {}, e = {}, inc = {}", orbit.a, orbit.e, orbit.inc);
```

Integrate

```
reb_simulation_integrate(r, 1000.0); // integrate until time t = 1000
reb_simulation_steps(r, 10_000); // or: take exactly 10000 steps
```

Check accuracy with energy

```
let e0 = reb_simulation_energy(r);
reb_simulation_integrate(r, 1000.0);
```

```
let e1 = reb_simulation_energy(r);
println!("relative energy drift: {e}", ((e1 - e0) / e0).abs());
```

Do something during the run (heartbeat)

```
fn my_heartbeat(r: &mut reb_simulation) {
    if reb_simulation_output_check(r, 10.0) { // every 10 time units
        println!("t = {}", r.t);
    }
}
r.heartbeat = Some(my_heartbeat);
```

Turn on collisions

```
r.collision = REB_COLLISION::DIRECT;
r.collision_resolve = Some(reb_collision_resolve_hardsphere);
```

Particles need a physical radius `p.r` for this to do anything.

Save and reload a simulation

Files are interchangeable with C-REBOUND: a file written here loads in the C program and vice versa.

```
reb_simulation_save_to_file(r, Some("run.bin")); // save a snapshot
reb_simulation_save_to_file_interval(r, "run.bin", 10.0); // auto-save every 10 time units

let mut restored = reb_simulation_create_from_file("run.bin", -1).unwrap(); // -1 = latest
reb_simulation_integrate(&mut restored, 2000.0);
```

Watch it in a browser

```
reb_simulation_start_server(r, 1234); // then open http://localhost:1234
reb_simulation_integrate(r, 1e6);
reb_simulation_stop_server(r);
```

The first call downloads `rebound.html` if it is not already in the folder.

Set options that live on the integrator

Some settings belong to a specific integrator, so you reach them like this:

```
if let reb_integrator_state::whfast(ref mut wh) = r.integrator {
    wh.corrector = 17; // higher-order symplectic corrector
    wh.safe_mode = 0; // faster; combines steps
}

if let reb_integrator_state::ias15(ref mut ias) = r.integrator {
    ias.epsilon = 1e-9; // accuracy target
}
```

7. Adding extra physics with REBOUNDx

Add the second dependency to Cargo.toml:

```
[dependencies]
rebound_rs = { path = "C:/work/rebound_rust" }
reboundx_rs = { path = "C:/work/reboundx_rust" }
```

The pattern is always the same:

1. `rebx_attach` — connect REBOUNDx to the simulation
2. `rebx_load_force` — pick an effect by name
3. `rebx_add_force` — switch it on
4. `rebx_set_param_*` — set the numbers the effect needs

Example: general-relativistic precession

```
use rebound_rs::*;
use reboundx_rs::*;

rebx_attach(&mut sim);
let gr = rebx_load_force(&mut sim, "gr_potential").unwrap();
rebx_add_force(&mut sim, gr);

if let Some(rebx) = rebx_extras_mut(&mut sim) {
    // the speed of light, in the same units as the simulation
    rebx_set_param_double(rebx, rebx_ap::force(gr), "c", 10065.32);
}

reb_simulation_integrate(&mut sim, 1000.0);
```

Where parameters live

In C you write `&sim->particles[0].ap` to mean "particle 0's parameter list". Here you write `rebx_ap::particle(0)`. The three possibilities:

Meaning	C	Rust
a particle's parameters	<code>&sim->particles[i].ap</code>	<code>rebx_ap::particle(i)</code>
a force's parameters	<code>&force->ap</code>	<code>rebx_ap::force(idx)</code>
an operator's parameters	<code>&operator->ap</code>	<code>rebx_ap::operator_(idx)</code>

Example: tides and spin

This is the most involved effect, and the one used in our acceptance tests. A body "has structure" (can be distorted by tides) once you give it a radius, a Love number k_2 , a moment of inertia I and a spin vector Ω .

```
rebx_attach(&mut sim);
let tides = rebx_load_force(&mut sim, "tides_spin").unwrap();
rebx_add_force(&mut sim, tides);
```

```

if let Some(rebx) = rebx_extras_mut(&mut sim) {
    // the star
    rebx_set_param_double(rebx, rebx_ap::particle(0), "k2", 0.07);
    rebx_set_param_double(rebx, rebx_ap::particle(0), "I",
        0.07 * solar_mass * solar_rad * solar_rad);
    rebx_set_param_vec3d (rebx, rebx_ap::particle(0), "Omega",
        reb_vec3d { x: 0., y: 0., z: solar_spin });
    rebx_set_param_double(rebx, rebx_ap::particle(0), "tau", solar_tau);
}

```

Reading a parameter back

```

if let Some(rebx) = rebx_extras_ref(&sim) {
    if let Some(omega) = rebx_get_param_vec3d(rebx, rebx_ap::particle(1), "Omega") {
        println!("spin = ({}, {}, {})", omega.x, omega.y, omega.z);
    }
}

```

A getter returns `None` in exactly the cases where the C returns a null pointer — i.e. the parameter was never set.

The available effects

Kind	Names you can pass to <code>rebx_load_force</code> / <code>rebx_load_operator</code>
Relativity	<code>gr</code> , <code>gr_full</code> , <code>gr_potential</code> , <code>lense_thirring</code>
Tides & spin	<code>tides_spin</code> , <code>tides_constant_time_lag</code> , <code>tides_dynamical</code>
Migration & disks	<code>modify_orbits_forces</code> , <code>modify_orbits_direct</code> , <code>type_I_migration</code> , <code>exponential_migration</code> , <code>gas_damping_timescale</code> , <code>gas_dynamical_friction</code>
Radiation	<code>radiation_forces</code> , <code>yarkovsky_effect</code>
Gravity shape	<code>gravitational_harmonics</code> , <code>central_force</code>
Other	<code>stochastic_forces</code> , <code>modify_mass</code> , <code>track_min_distance</code> , <code>integrate_force</code>

Each effect's required parameters are documented as Rust doc comments at the top of its module, carried over from the C source. To read them:

```

cd C:\work\reboundx_rust
cargo doc --open

```

8. Every build command, in one place

From `C:\work\rebound_rust`:

```

cargo build --release
cargo test --release
cargo clippy --release --all-targets
cargo doc --no-deps --open

cargo build --release --example shearing_sheet
cargo build --release --example shearing_sheet_test

```

```
cargo build --release --example integrators_test
cargo build --release --example libm_diff
cargo build --release --example bs_pow_diff
cargo build --release --example derivatives_test
cargo build --release --example frequency_test
cargo build --release --example archive_test
cargo build --release --example server_test
cargo build --release --example addfmt_test
```

From C:\work\reboundx_rust:

```
cargo build --release
cargo test --release
cargo clippy --release --all-targets
cargo doc --no-deps --open

cargo build --release --example tides_spin_pseudo
cargo build --release --example tides_spin_kozai
cargo build --release --example tides_spin_migration
```

To run any example:

```
cargo run --release --example <name>
cargo run --release --example integrators_test -- whfast 2 500    # with arguments
```

Every one of these completes with **zero warnings**. Nothing is downloaded: both crates are **std-only**, so they build with no network connection.

Part III — Provenance: how it was made and checked

9. The machine and the tools

Everything below was done on one computer, with no Linux, no WSL2, no GCC and no Clang involved at any point.

Component	Value
Operating system	Windows 11 Pro for Workstations 10.0.26200, x86-64
C compiler	MSVC cl 19.51.36256 for x64 (Visual Studio 2026 Build Tools)
Compiler environment script	C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
make	GnuWin32 Make 3.81
vcpkg	C:\Users\youruser\vcpkg\vcpkg.exe
Rust	rustc 1.91.1 (ed61e7d7e 2025-11-07), cargo 1.91.1, target x86_64-pc-windows-msvc
REBOUND source	github.com/hannorein/rebound, 5.1.1 , commit dad5f97806ecbb408dcaff728851c64e67f9f6eb
REBOUNDx source	github.com/dtamayo/reboundx, 5.1.0

GLFW (for completeness)

REBOUND's native 3-D viewer uses a library called GLFW on Linux and macOS. It was installed for future use:

```
C:\Users\youruser\vcpkg\vcpkg.exe install glfw3:x64-windows
```

This gave glfw3 **3.5.1** (C:\Users\youruser\vcpkg\installed\x64-windows\lib\glfw3dll.lib). However, **REBOUND's own build system forces OpenGL off on Windows** (src/Makefile.defs line 21 prints "OpenGL not supported on Windows. Setting OPENGL=0"), so the C reference does not link GLFW, and visualisation is done through the built-in web server instead — which *is* ported. The vcpkg install is staged for any future work on the OpenGL path.

10. Building the C originals (the reference)

To prove our Rust gives the same answers, we first need the originals to compare against.

10a. REBOUND

```
cd C:\work
git clone https://github.com/hannorein/rebound.git rebound\rebound

cd C:\work\rebound\rebound\examples\shearing_sheet
cmd /c 'set PATH=C:\Program Files (x86)\GnuWin32\bin;%PATH% && "C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" && make'
```

Two Windows traps. 1. GnuWin32 must be added to PATH **before** vcvars64.bat runs, exactly as written. Windows expands %PATH% when it *reads* the line, so doing it the other way round silently erases what vcvars64.bat just set up, and you get "cl is

not recognized". 2. Compile C **only** from PowerShell via `cmd /c '...`. Git-Bash's `cmd //c` can fail *silently*, leaving a stale `.exe` behind — so you unknowingly test an old build. This cost us real debugging time.

Each file compiles as:

```
cl -c /DBUILDINGLIBREBOUND /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /D_CRT_NONSTDC_NO_WARNINGS /Ox /fp:precise -DSERVER -DGITHASH=dad5f97806ecbb408dcaff728851c64e67f9f6eb /Fo:rebound.obj rebound.c
```

`/Ox` optimises for speed; `/fp:precise` forbids the compiler from rearranging floating-point arithmetic, which is essential for a bit-exact comparison.

For linking test harnesses we also need a *static* library containing the internal (non-exported) functions:

```
cd C:\work\rebound\rebound\src
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && lib /nologo /OUT:librebound_static.lib *.obj'
```

One detail worth recording: on Windows, REBOUND vendors glibc's `rand_r` random-number generator directly in `rebound.c` (three rounds of `seed = seed*1103515245 + 12345`, `REB_RAND_MAX = 2147483647`). That is why random initial conditions are identical across platforms — and why our Rust reproduces them exactly.

10b. REBOUNDx

```
cd C:\work\reboundx\src
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && cl /nologo /c /I"..\..\rebound\rebound\src" /I"." /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /D_CRT_NONSTDC_NO_WARNINGS /DLIBREBOUNDX /Ox /fp:precise *.c'
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && lib /nologo /OUT:libreboundx.lib *.obj'
```

Two files do not compile under MSVC, and this is worth explaining because it is the only genuine portability problem we hit in either project.

`gr_full.c` and `interpolation.c` use **C99 variable-length arrays** — arrays whose size is decided while the program runs:

```
double a_const[N][3];      /* gr_full.c */
double u[n];               /* interpolation.c */
```

Microsoft's C compiler has never supported these. GCC and Clang do.

Our fix changes **only where the memory comes from** (the heap instead of the stack), never any arithmetic:

```
double (*a_const)[3] = malloc((size_t)N*sizeof(*a_const));
double* u = malloc((size_t)n*sizeof(double));
```

with matching `free()` calls so the lifetime matches the original. Same element type, same indices, same values, same order — the numbers are untouched. The patched copies live in `reboundx_rust/porttest/msvc_shim/` so that **the upstream source is never modified**, and their object files are then included in the library:

```
cd C:\work\reboundx_rust\porttest\msvc_shim
```



```
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
" >nul && cl /nologo /c /I"..\..\..\rebound\rebound\src" /I"..\..\..\reboundx\src" /D_GNU_SOURCE
/D_CRT_SECURE_NO_WARNINGS /D_CRT_NONSTDC_NO_WARNINGS /DLIBREBOUNDX /Ox /fp:precise gr_full.c
interpolation.c'
```

Note this affects the **C reference build only**. The Rust port has no such problem: Rust's `Vec` is a growable array and needs no shim.

11. How the translation was done

The rules, applied to every file in both projects:

1. **Fidelity first.** Control flow, constants, tolerances and arithmetic *order* match the C expression by expression. Floating-point addition is not associative: $(a+b)+c$ and $a+(b+c)$ can give different answers, so the C's exact bracketing is transcribed even where it looks redundant.
2. **Zero unsafe, zero dependencies, zero warnings.** Enforced by `#![forbid(unsafe_code)]` and `#![deny(warnings)]` at the top of each crate.
3. **C names are kept.** `reb_simulation_create`, `rebx_add_force`, `reb_particle` and so on, so C code and documentation read across directly.
4. **Missing symbols are reported, never invented.** If something could not be translated it is listed in section 18, not silently faked.

The one structural problem, and its solution

C code passes raw pointers around. Safe Rust does not allow two pieces of code to hold a mutable pointer to the same thing at once. REBOUND does exactly that: a step function receives both the simulation and the integrator's own state, which *lives inside* the simulation.

The consistent solution, used everywhere: **take the state out, use it, put it back.**

```
let mut state = match std::mem::replace(&mut r.integrator, reb_integrator_state::none) {
    reb_integrator_state::whfast(s) => s,
    other => { r.integrator = other; return; }
};
// ... now both 'r' and 'state' can be used freely ...
r.integrator = reb_integrator_state::whfast(state);
```

This is exactly equivalent to what the C does, and it is checked by the compiler. The same pattern carries REBOUNDx's state through `reb_simulation::extras`.

12. File-by-file accounting: REBOUND

All 31 C translation units, and what happened to each. **29 Rust modules, 19,075 lines.**

C file	Rust module	Status
rebound.c	tools.rs, server.rs, lib.rs	Ported: messages, reb_exit, reb_strcmp_ignore_whitespace, reb_check_fp_contract, favicon data, version/githash. Not applicable in Rust: malloc wrappers, SIGINT handler, custom-integrator registry, AVX-512 detection, OpenMP setter.
simulation.c	simulation.rs	Ported + verified
particle.c	particle.rs	Ported + verified
tools.c	tools.rs	Ported + verified (RNG family, energies, centre-of-mass, orbit conversions incl. Pal coordinates, add_fmt + Solar System data, MEGNO)
boundary.c	boundary.rs	Ported + verified (incl. shear ghost boxes)
tree.c	tree.rs	Ported + verified (octree as an index arena)
gravity.c	gravity.rs	Ported + verified (basic, compensated, tree, jacobi, variational)
collision.c	collision.rs	Ported + verified (all searches and resolvers)
output.c	output.rs	Ported (screenshot excluded with the display subsystem)
transformations.c	transformations.rs	Ported + verified
rotations.c	rotations.rs	Ported + verified
derivatives.c	derivatives.rs	Ported + verified — all 65 functions
frequency_analysis.c	frequency_analysis.rs	Ported + verified (MFT / FMFT / FMFT2)
integrator_none/ser/heap/rogue/ias15/whfast512	integrator.rs	Ported + verified (IAS15 encounter machinery, hooks)
integrator_mercurius	integrator_mercurius.rs	Ported + verified (full reb_ode framework)
integrator_bs.c	integrator_bs.rs	Ported + verified (reversible checks, BS/IAS15 encounter and pericentre paths)
integrator_trace	integrator_trace.rs	Ported + verified (reversible checks, BS/IAS15 encounter and pericentre paths)
integrator_whfast512	integrator_whfast512.rs	Ported + verified (reversible checks, BS/IAS15 encounter and pericentre paths)
integrator_windows512	integrator_windows512.rs	Ported + verified (reversible checks, BS/IAS15 encounter and pericentre paths)
binarydata.c	binarydata.rs	Ported + verified (same byte format)
simulationarchive	simulationarchive.rs	Ported + verified
server.c	server.rs	Ported + verified (HTTP endpoints, base64, threading adapted)
fmemopen.c	—	Not applicable: replaced by std::io::Cursor
display.c, glad.c	—	Excluded: OpenGL (the Windows C build excludes it too)
communication_mpi.c	—	Excluded: MPI (not in the Windows C build)

About WHFast512. Its fast core is hand-written AVX-512 assembly compiled only by GCC/Clang. Under MSVC the C compiles the `#else // Not 64 bit, Windows + cl` branch, which contains only stubs that report "AVX512 is not supported on your platform." Our Rust reproduces exactly that reference behaviour — so on Windows, both integrate nothing, identically.

13. File-by-file accounting: REBOUNDx

All 33 C translation units (8,452 lines of C, including headers) become 34 Rust modules (8,798 lines). Every .c file is accounted for below.

Core machinery

C file	Lines	Rust module	Lines	Status
core.c	1186	core.rs	1494	Ported. All 107 default parameter registrations, in the same order with the same types (verified by direct diff against the C). <code>rebx_load_force/rebx_load_operator</code> carry the complete name $\rightarrow$ function tables.
rebxtools.c	291	rebxtools.rs	175	Ported (com/jacobi helpers, <code>rebx_tools_spin_angular_momentum</code> , <code>rebx_simulation_irotate</code>).
linkedlist.c	106	—	—	Not applicable. These are the linked-list helpers (<code>rebx_add_node</code> , <code>rebx_remove_node</code> , <code>rebx_len</code>). In Rust the lists are <code>Vecs</code> , so the helpers become ordinary vector operations. The C's <i>prepend</i> order is preserved by inserting at index 0 — see deviation 8 in section 14.
output.c	293	output.rs	640	Binary serialization. Verified against the real C library in both directions — see §15.10.
input.c	732	input.rs	1097	Binary deserialization. Reads files written by the C, and vice versa.

Forces

C file	Lines	Rust	Lines
tides_spin.c	418	tides_spin.rs	643
gr_full.c	393	gr_full.rs	459
tides_dynamical.c	346	tides_dynamical.rs	409
gravitational_harmonics.c	318	gravitational_harmonics.rs	454
stochastic_forces.c	252	stochastic_forces.rs	385
gr.c	250	gr.rs	276
yarkovsky_effect.c	248	yarkovsky_effect.rs	418
tides_constant_time_lag.c	228	tides_constant_time_lag.rs	262
type_I_migration.c	214	type_I_migration.rs	235
gas_dynamical_friction.c	176	gas_dynamical_friction.rs	205
central_force.c	152	central_force.rs	178
gas_damping_timescale.c	141	gas_damping_timescale.rs	181
modify_orbits_forces.c	139	modify_orbits_forces.rs	178
radiation_forces.c	126	radiation_forces.rs	156
gr_potential.c	120	gr_potential.rs	131
lense_thirring.c	119	lense_thirring.rs	129
exponential_migration.c	112	exponential_migration.rs	134
inner_disk_edge.c	77	inner_disk_edge.rs	56

Operators, steppers and REBOUNDx's own integrators

C file	Lines	Rust	Lines
interpolation.c	180	interpolation.rs	263

C file	Lines	Rust	Lines
<code>modify_orbits_direct.c</code>	144	<code>modify_orbits_direct.rs</code>	165
<code>steppers.c</code>	130	<code>steppers.rs</code>	180
<code>integrator_implicit_midpoint.c</code>	124	<code>integrator_implicit_midpoint.rs</code>	210
<code>track_min_distance.c</code>	98	<code>track_min_distance.rs</code>	112
<code>integrator_rk4.c</code>	90	<code>integrator_rk4.rs</code>	243
<code>integrate_force.c</code>	78	<code>integrate_force.rs</code>	102
<code>modify_mass.c</code>	74	<code>modify_mass.rs</code>	68
<code>integrator_rk2.c</code>	66	<code>integrator_rk2.rs</code>	154
<code>integrator_euler.c</code>	41	<code>integrator_euler.rs</code>	44

Plus `types.rs` (270 lines), which has no single C counterpart: it holds the structures from `reboundx.h` together with the three substitutions that replace the C’s pointer graph (documented at the top of that file).

14. Deviations from the C, and why each is safe

Every difference is mechanical — a consequence of Rust’s ownership rules — and none changes a computed number.

1. **Ownership instead of pointers.** `r->particles` (a `malloc`’d block) becomes a `Vec<reb_particle>`. The octree’s individually allocated cells become an index arena, rebuilt exactly as the C rebuilds cells. Particle names, which the C interns as `char*`, become an index into a name list. The `ap` and `sim` back-pointers inside `reb_particle` are dropped — functions that used them take the simulation explicitly.
2. **Integrator state.** The C’s `void* state` behind a function-pointer table becomes an `enum` with one variant per integrator, taken out and put back around each step (section 11).
3. **`r->map` aliasing.** MERCURIUS and TRACE make `r->map` point at their own `encounter_map` — one array with two names. The Rust *moves* the vector in and out instead. Same contents at every observable point.
4. **Binary file pointers.** A `reb_particle` is written in its exact 112-byte C memory layout. The C stores real heap addresses in the `name/ap/sim` slots; those addresses are only ever compared for equality against addresses stored beside the name list. Rust writes 0 for `ap/sim` and a synthetic id for `name`, reproducing that protocol exactly — which is why archive files are interchangeable in both directions (verified, §15.5).
5. **Server threading.** The C server thread dereferences the simulation directly under a mutex. Safe Rust cannot share `&mut` across threads, so the same handshake is expressed with a shared snapshot and key queue, serviced by the integration loop at exactly the points where the C locks and unlocks. The HTTP behaviour is unchanged (verified, §15.6).
6. **Variadic arguments.** `reb_simulation_add_fmt(r, fmt, ...)` takes its values as an ordered `&[reb_fmt_arg]` slice, consumed token by token like `va_arg`.
7. **REBOUNDx parameters.** The C stores `void*` value plus a type tag and casts on read. Rust fuses the two into one `enum`, so a wrong-type read is impossible instead of undefined. For correct programs the behaviour is identical; for incorrect ones Rust reports rather than corrupts.

8. **REBOUNDx list order.** The C's `rebx_add_node` *prepends* to its linked lists, so a list iterates in reverse insertion order — and that order decides the order accelerations are summed, which changes floating-point results. The Rust lists are `Vecs` whose **index 0 is the head**, and the add helper inserts at 0. Iteration order is identical, element for element.
9. **Removed platform branches.** OpenMP `#pragma` blocks, `reb_sigint` polling and MPI branches are omitted — none of them is compiled in the C reference build either.
10. **Uninitialised C memory.** Where C leaves struct members uninitialised (e.g. `reb_orbit_nan`), Rust zero-initialises before setting the same members.
11. **A C quirk preserved in spirit.** `rebx_additional_forces` declares `const double N = sim->N`; — a *double* — then passes it to a function taking an `int`. For any realistic particle count this conversion is exact, so the Rust uses an integer directly.
12. **The git-hash stamp in REBOUNDx binary files.** The header of a REBOUNDx binary file carries a 26-byte field recording which source revision the library was compiled from. The C fills it in from `git` at compile time via its makefile, and writes the literal `notavailable0000000000000000` when git information is not available — which is the case for the reference build used here. The Rust crate is built by `cargo` and has no such compile-time git step, so it writes 26 zero bytes. This is the **only** difference between a C-written and a Rust-written binary file (26 bytes out of 6,392 in the round-trip test), it contains no simulation data, and neither library reads it back for any purpose. Both libraries read each other's files correctly regardless (verified, §15.9).

15. The complete verification record

The method throughout: run the identical experiment in the MSVC-compiled C and in Rust, dump every value as raw IEEE-754 bits, and compare byte for byte. A run passes only if **every bit of every value** matches.

The record at a glance

What was checked	Scale	Result
Maths library agreement (§15.0)	200,000 samples $\times$ 21 functions	20 of 21 exact; <code>pow</code> differs, §16
Integrator matrix (§15.1)	63 configurations $\times$ 500 steps	63/63 bit-identical
Shearing sheet (§15.2)	1,482 particles, 400 steps, 102,533 collisions	byte-identical, SHA-256 75bdaab7...
Orbital derivatives (§15.3)	65 functions	130/130 outputs bit-identical
Frequency analysis (§15.4)	MFT, FMFT, FMFT2	bit-identical
Simulation archive (§15.5)	C $\rightarrow$ Rust and Rust $\rightarrow$ C continuations	bit-identical both ways
Web server (§15.6)	blob served by Rust, read by C	bit-identical state
<code>add_fmt</code> and datasets (§15.7)	all format tokens	bit-identical

What was checked	Scale	Result
REBOUNDx tides_spin (§15.8)	3 examples × short and long runs	6/6 bit-identical
REBOUNDx binary files (§15.9)	round trip, both directions	6,366 of 6,392 bytes identical; only the git-hash stamp differs
Automated test suite — REBOUND	394 tests	394 pass, 0 fail
Automated test suite — REBOUNDx	137 tests	137 pass, 0 fail
Compiler and clippy warnings (§17)	both crates, all targets	zero

The test suites found three genuine translation defects, which are written up honestly in §15.10 rather than quietly fixed.

To run both suites yourself:

```
cd C:\work\rebound_rust
cargo test --release
```

```
cd C:\work\reboundx_rust
cargo test --release
```

15.0 The foundation: which maths functions agree?

Before anything else we established what the two languages’ maths libraries do, using a differential harness of 200,000 samples per function:

```
cd C:\work\rebound_rust
cargo build --release --example libm_diff
cd porttest
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
" >nul && cl /nologo /Ox /fp:precise libm_diff.c /Fe:libm_diff.exe'
.\libm_diff.exe
..\target\release\examples\libm_diff.exe
```

Result: on x86_64-pc-windows-msvc, Rust and Microsoft’s C library are **bit-identical** for `sin`, `cos`, `tan`, `atan2`, `sqrt`, `fmod`, `exp`, `log` and `cbrt`. **pow is the single divergent function** — see section 16.

This is what makes true bit-identity testing possible at all.

15.1 The integrator matrix — 63 configurations

A fixed three-body problem (star of mass 1; planet of mass 10^{-3} at $x = 1.6$ with $v_y = 0.5$; moon of mass 10^{-7} at $x = 1.7$, $v_y = 0.6$, $z = 0.01$, $v_z = 0.001$), $G = 1$, $dt = 0.01$, 500 steps, final state dumped as raw bits.

Build the C harness once:

```
cd C:\work\rebound_rust\porttest
```

```
cmd /c "C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
" >nul && cl /nologo /I"...\rebound\rebound\src" /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /
D_CRT_NONSTDC_NO_WARNINGS /Ox /fp:precise integrators_test.c librebound.lib /Fe:integrators_test.
exe'
```

To check a single configuration, give the harness a configuration name, a leapfrog order (ignored by the others) and a step count, then compare:

```
cd C:\work\rebound_rust\porttest
.\integrators_test.exe whfast 2 500
..\target\release\examples\integrators_test.exe whfast 2 500
Compare-Object (Get-Content state_c_final.txt) (Get-Content state_rust_final.txt)
```

Compare-Object printing nothing means that configuration matched.

To check **all 63 in one go**, run the sweep script that ships in `porttest\`. It knows every configuration name, runs each one in both languages, compares the dumps and prints a tally:

```
cd C:\work\rebound_rust\porttest
powershell -ExecutionPolicy Bypass -File .\run_integrator_matrix.ps1 500
```

It takes a few minutes and ends with:

```
63 of 63 configurations bit-identical.
ALL CONFIGURATIONS BIT-IDENTICAL
```

Result: 63 of 63 configurations bit-identical.

One thing to know before you run it: **every harness in porttest\ writes its results to the same two filenames**, `state_c_final.txt` and `state_rust_final.txt`. That is deliberate — it keeps the comparison command identical for every test — but it means each run overwrites the last one's dumps, and the sweep script above consumes them. If you run the sweep and then try to check the shearing sheet, you will find the shearing sheet's files gone. Just re-run that pair to recreate them:

```
cd C:\work\rebound_rust\porttest
.\rebound_test.exe 400
..\target\release\examples\shearing_sheet_test.exe 400
```

(`rebound_test.exe` is the compiled `problem_test.c`, the C side of the shearing-sheet test.)

Integrator	Configurations tested (all identical)
none	none
ias15	ias15 (plus a separate 1000-step adaptive run)
leapfrog	orders 2, 4, 6, 8
whfast	default, c11, c17(+corrector2), dh, whds, bary, mk, comp, lazy, usafe
saba	default, 1, 2, 3, 4, cm2, c12, 104, 864, h844, h864, h1064, usafe
janus	default (6), 2, 4, 8, 10
eos	default, all nine φ diagonals, 2-7, 5-8, usafe
mercurius	default, usafe, c4, c5, inf, hill01 — the default keeps the moon inside the planet's critical radius, so the IAS15 close-encounter machinery runs on every step (also verified to 5000 steps)
bs	default, tight (10^{-11}), loose (10^{-6}), maxdt

Integrator	Configurations tested (all identical)
trace	default, pbs, ias15, hill1, perinone, eta001 — cross-checked to genuinely take the BS-encounter and pericentre paths

This matrix was re-run in full after every change to the crate, including after adding the `extras` field needed by REBOUNDx. It has always been 63/63.

15.2 The shearing sheet — the acceptance test

The stock Saturn’s-rings example: SEI integrator, octree gravity, tree collision search, shear-periodic boundary, hard-sphere collisions with the Bridges bounce law, and `rand_r` initial conditions. Seed 42, **1,482 particles**, 400 steps, **102,533 collisions**.

```
cd C:\work\rebound_rust\porttest
.\rebound_test.exe 400
..\target\release\examples\shearing_sheet_test.exe 400
```

```
Get-FileHash state_c_final.txt -Algorithm SHA256
Get-FileHash state_rust_final.txt -Algorithm SHA256
```

Result: byte-identical, matching SHA-256

```
75BDAAB7109F125192F56AEB0CCDCAC554AF88A165FE11075EC5A871178521F0
```

Timing on this machine: C 2.2 s, Rust 1.7 s.

This test did not pass on the first attempt. 330 particles had drifted, and the cause turned out to be `pow` — the one library function where Rust and Microsoft’s C disagree. The investigation is worth summarising, because it is the clearest illustration of how a 1-ULP difference becomes a visible one.

The two runs stayed identical for 77 steps and separated at step 78, in the collision between particles 390 and 1,456. The Bridges coefficient-of-restitution law computes $0.32 * \text{pow}(v, -0.234)$. At that one impact speed Rust’s `pow` and Microsoft’s differed in the last bit — about one part in 10^{16} . That changed the bounce by one bit, which changed where both particles were on the next step, which changed which tree cells they landed in, which changed the *set* of collisions found, and from there the two runs diverged completely.

The fix was not to change any physics. Rewriting the identical formula as `exp(-0.234 * log(v))` on **both** sides — C and Rust — removes the only divergent function from the calculation, and the entire run becomes bit-identical. `exp` and `log` agree exactly on this platform; only `pow` does not. This is covered in full in section 16.

Two Windows-specific traps also cost time here and are worth knowing: the C example seeds its generator from the clock and process id (pinned to seed 42 for the comparison), and the stock example never terminates, which is why the comparison uses the `_test` variants on both sides.

15.3 Orbital derivatives — 65 functions

All 65 `reb_particle_derivative_*` functions, over two independent particle/primary configurations. **Result: 130/130 output lines bit-identical.**

15.4 Frequency analysis

A synthetic three-frequency signal (0.30, 0.55, 0.11 radians per sample), 256 samples, analysed in all three modes. **Result: MFT, FMFT and FMFT2 all bit-identical.** This exercises the FFT, the Hanning window, golden-section maximisation, Gram-Schmidt orthogonalisation and the amplitude sort end to end.

15.5 Simulationarchive — cross-language round trips

The strongest interoperability test. One implementation runs 3×100 steps, saving a snapshot after each 100. The *other* implementation loads snapshot 1 (the 200-step state), continues for 100 more steps, and must land on the writer's 300-step state **bit-exactly**.

```
cd C:\work\rebound_rust\porttest
.\archive_test.exe whfast-usafe write
..\target\release\examples\archive_test.exe whfast-usafe continue
..\target\release\examples\archive_test.exe whfast-usafe write
.\archive_test.exe whfast-usafe continue
```

Result: identical in all four directions, for `whfast-usafe` (which round-trips unsynchronised Jacobi coordinates) and `ias15` (which round-trips the adaptive-step restart arrays). Archive files, including the incremental diff-blob append format, are fully interchangeable between the C and Rust builds.

Why the .bin files are not committed

Run the commands above and `porttest\` fills with `archive_c_*.bin` and `archive_rust_*.bin`. Those files are **not** kept in this repository, and the reason is worth knowing, because it is a real difference between the two languages that the rest of this document does not otherwise show.

A C `struct` usually contains *padding* — unused bytes the compiler inserts so that each member starts at an address the processor likes. When C writes a whole struct to a file in one go, it writes the padding too, and nothing requires the padding to have been set to anything. It holds whatever happened to be in that memory beforehand.

Inspecting the committed `archive_c_whfast-usafe.bin` showed exactly that. Six times over, sitting between two perfectly ordinary floating-point numbers, was a fragment of text that had nothing to do with the simulation:

```
AppData\Local\pnpm;C:\Users\...\lmstudio\bin;C:\Users\
```

That is a piece of the `PATH` environment variable of the machine that wrote the file — leftover heap memory, captured and published by accident. It is harmless here, but the same mechanism could just as easily have caught something that mattered.

The Rust port does not do this. Safe Rust has no uninitialised memory: the port builds its output buffer explicitly, so its padding bytes are zero. The `archive_rust_*.bin` files contain no such fragments — checked, and they are clean.

So the `.bin` files are treated as what they are — regenerable output — and `porttest/*.bin` is in `.gitignore`. Nothing is lost: every command needed to recreate them is printed above, and the comparison is the point, not the files.

15.6 The web server

The Rust example pauses a 100-step simulation and serves it over HTTP; the blob is then loaded by the C build.

```
cd C:\work\rebound_rust\porttest
# in one terminal:
..\target\release\examples\server_test.exe
# in another:
curl.exe -s http://localhost:12873/simulation --output served.bin
curl.exe -s http://localhost:12873/keyboard/81
.\archive_test.exe whfast load served.bin
```

Result: the C build loads the Rust-served blob to the bit-identical state (3,448 bytes, header REBOUND Binary File. Version: 5.).

15.7 add_fmt and the built-in datasets

Built-in "solar system" dataset plus particles created from orbital elements, from Pal coordinates, and from an orbital period. **Result: all 12 particles bit-identical.**

15.8 REBOUNDx: the tides_spin acceptance tests

Three of REBOUNDx's own shipped examples were run in both languages and compared bit for bit. Together they exercise the tidal and spin forces, the spin differential-equation machinery, general-relativistic precession, migration forces, both a fixed-step and an **adaptive** integrator, the rotation into the invariable plane, and changing a parameter mid-run.

```
cd C:\work\reboundx_rust\porttest
.\tides_spin_pseudo_c.exe 62.83185307179586 2>$null
..\target\release\examples\tides_spin_pseudo.exe 62.83185307179586
```

then in PowerShell:

```
Compare-Object (Get-Content state_pseudo_c.txt) (Get-Content state_pseudo_rust.txt)
```

Result: bit-identical in all six runs.

Test	What it exercises	End time	Result
tides_spin_pseudo_synchronous	Williston	10 orbits	identical
"	"	100 orbits	identical
tides_spin_kozai	adaptive IAS15 , tides_spin + gr_potential	t = 1,000	identical
"	"	t = 100,000 (full default)	identical
tides_spin_migration_driven	WHFast, tides_spin + modify_orbits_forces, mid-run parameter change	10 orbits	identical
"	"	100 orbits	identical

Every position, velocity, mass **and spin vector** matched to all 64 bits.

The Kozai result is the strongest of the three: because IAS15 chooses its own step size, and those choices depend on the REBOUNDx forces, matching bit for bit at $t = 100,000$ means both programs took **the identical sequence of thousands of adaptive steps** — not merely that they arrived at the same place.

Every command needed to build the C reference — including the MSVC portability shim — is given in full in section 10 and section 19 of this document. (The same three tests are also written up on their own, with a longer narrative, in `reboundx_port_test.md`; you do not need that file, because everything required to reproduce the result is here.)

15.9 REBOUNDx binary files: C writes, Rust reads, and the reverse

REBOUNDx can save a whole configuration — every force, every operator, every operator step and every parameter attached to every particle — to a binary file. If the Rust wrote that file even slightly differently, the two libraries could no longer exchange data, and a saved simulation would be trapped in whichever language created it.

So both libraries were made to build the *same* configuration and write it out: two forces (`gr_potential`, `central_force`), two operators (`modify_mass`, `drift`) across three operator steps with pre- and post-timestep timings, and parameters of every supported type — `double`, `int` (including a negative one), `uint32`, `vec3d`, `string`, and a pointer to a force.

```
cd C:\work\reboundx_rust\porttest
..\target\release\examples\rebx_binary_roundtrip.exe          # Rust writes rebx_binary_roundtrip.bin
.\rebx_binary_roundtrip_c.exe rebx_c_reference.bin             # C writes rebx_c_reference.bin
```

Compare the two files byte by byte:

```
$a=[IO.File]::ReadAllBytes("rebx_binary_roundtrip.bin")
$b=[IO.File]::ReadAllBytes("rebx_c_reference.bin")
"rust=$(($a.Length)  c=$(($b.Length))"
$d=0; for($i=0;$i -lt $a.Length;$i++){ if($a[$i] -ne $b[$i]){ $d++ } }
"differing bytes: $d"
```

Result: both files are 6,392 bytes, and 6,366 of those bytes are identical.

The 26 bytes that differ are offsets 37–62 — a single field in the file header, and it contains no simulation data at all. It is the *git hash*: a stamp recording which exact revision of the source the library was compiled from. The C build had no git information available when it was compiled, so it writes the literal text `notavailable0000000000000000`; the Rust crate is not built through a git-aware makefile at all, so it writes 26 zero bytes. This is discussed as a deliberate deviation in section 14. **Every byte of actual physics — all masses, positions, velocities, parameters, names, types and orderings — matches.**

Identical bytes are the strong result, but the practical question is whether each library can *read* what the other wrote. Both directions were tested:

```
# Rust reads the file the C wrote, then re-serializes it
..\target\release\examples\rebx_binary_roundtrip.exe rebx_c_reference.bin

# C reads the file Rust wrote, and prints everything it recovered
.\rebx_binary_read_c.exe rebx_binary_roundtrip.bin
```

Direction	Result
Rust reads the C's file	25 / 25 checks passed , and re-serializing reproduces all 6,392 bytes
C reads the Rust's file	all 28 lines of recovered state identical to the C reading its own file

That last row is worth stating plainly: the C library was asked to read the Rust-written file and dump everything it found, then asked to do the same with its own file. The two dumps are identical line for line — including raw 64-bit patterns such as `p1.tau_mass 4636b0a8e891ffff`, the negative integer `p1.primary -12345`, the three components of the `Omega` vector, the string `p1.force gr_potential`, and the *order* the parameters come back in.

To reproduce that comparison:

```
.\rebx_binary_read_c.exe rebx_binary_roundtrip.bin 2>$null | Out-File -Encoding ascii
  readback_from_rust.txt
.\rebx_binary_read_c.exe rebx_c_reference.bin      2>$null | Out-File -Encoding ascii readback_from_c
  .txt
Compare-Object (Get-Content readback_from_rust.txt) (Get-Content readback_from_c.txt)
```

Compare-Object printing nothing means the files agree.

15.10 Three real defects the new test suite found

The Rust test suites (394 tests for REBOUND) were written from the C source rather than from the Rust, which is what made them able to find genuine translation defects. Three were found, confirmed against the C, and fixed. They are recorded here because "we tested it and found nothing" and "we tested it and found three things" are very different claims.

Defect 1 — the Kepler solver could hang forever

Symptom. A test of near-rectilinear (almost straight-line) hyperbolic motion never finished.

Diagnosis. For that motion the pericentre distance `q` is ~ 0 , so the bisection fallback's bounds become `X_max = dt/q = +inf` and `X_min = NaN`. The C's loop is

```
} while (fastabs(X_max-X_min) > fastabs((X_max+X_min)*1e-15));
```

With NaN, `A > B` is **false**, so the C exits after one pass. The Rust had translated the exit as

```
if fastabs(X_max - X_min) <= fastabs((X_max + X_min) * 1e-15) { break; }
```

and `A <= B` is **also false** for NaN — so it never breaks. `while (A > B)` and `if (A <= B) break` are *not* equivalent in the presence of NaN.

Fix. Negate the C's condition exactly: `if !(A > B) { break; }.`

Verification. A probe calling the solver directly in both languages (`porttest/kepler_rectilinear_c.c` and `examples/kepler_rectilinear.rs`) now agrees bit for bit at $h = 0$ and $h = 10^{-12}$, and at `dt = 0.1, 0.5, 2.0, 5.0` and `-2.0`. Before the fix, `dt = 2.0` hung in Rust and returned normally in C.

Follow-up. Every other `do { } while (float condition)` in the C was then audited for the same mistranslation. One more used the fragile idiom (the Plummer-sphere rejection loop in `tools.c`); it is provably NaN-free there, but it was rewritten in the faithful `!(a > b)` form anyway.

Defect 2 — archives reported major version 0

Symptom. A test asserting that a saved archive reports REBOUND's major version failed.

Diagnosis. The header reads "REBOUND Binary File. Version: 5.1.1". The C splits it at the : and the two .s, so the major-version substring is " 5" — **with a leading space**. C's `atoi(" 5")` skips leading whitespace and gives

1. The Rust helper collected a leading run of digits, hit the space first, and

gave 0. Minor and patch, which have no leading space, parsed correctly, which is why nothing else showed it.

Fix. Make the helper do what C's `atoi` does: skip whitespace, accept an optional sign, then take the digits.

Defect 3 — the variational centre-of-mass shift was wrong

Symptom. An audit comparing `tools.c` line by line flagged that the first-order variational `dm` accumulator summed the wrong array.

Diagnosis. The C is

```
for (size_t i=0;i<N;i++){ dm += particles[i+index].m; } /* REAL particles */
```

and the Rust summed `particles_var` instead. `dm` multiplies a whole term of the centre-of-mass shift, so for MEGNO/Lyapunov runs (where the variational masses are zero) that term vanished entirely in Rust and did not in C. A second, related problem: the whole **second-order** variational block was untranslated, and the early `return` that stood in for it also skipped the ordinary particle shift *and* the boundary check, leaving the simulation partly transformed.

Fix. Sum the real particle array, exactly as the C does — even though that looks like an upstream slip, because bit-exactness with C 5.1.1 is the whole point. Port the full second-order block, and restore the C's **two-pass** ordering (all second-order configurations before any first-order one, because the second-order shift reads the first-order particles pre-shift).

One case genuinely cannot be reproduced: for a first-order configuration with `index > 0`, the C reads `particles[i+index]` *past the end of the particle array*. That is undefined behaviour upstream. Safe Rust cannot reproduce whatever bytes happen to follow the array, so that case reports a clear error rather than inventing a value — in keeping with the project rule to report missing or unreproducible things, never to fake them.

Verification. A Sun+Jupiter MEGNO probe (`porttest/movetocom_var.c.c` and `examples/movetocom_var.rs`) now produces bit-identical output in both languages.

After the fixes

All three fixes touch code on the bit-identity paths, so the entire verification suite was re-run from scratch afterwards: **63/63 integrator configurations identical, the shearing sheet identical (same SHA-256), all three REBOUNDx acceptance tests identical, and 394 REBOUND tests passing with none ignored.**

16. The one known difference: `pow`

`pow(a, b)` — "a raised to the power b" — is the only maths function where Rust and Microsoft's C library disagree. Rust ships its own implementation rather than calling the system one.

How often, and by how much? Two independent measurements:

Measurement	Samples	Disagreements	Size
General sweep (<code>libm_diff</code>)	200,000	60 (0.030%)	≤ 2 ULP
BS-controller shapes (<code>bs_pow_diff</code>)	200,000	56 (0.0280%)	all exactly 1 ULP

Both implementations are correct to within what the C standard requires. They simply round a handful of cases differently.

Where it shows up in practice. We caught it twice, in completely different places, and both times the diagnosis was conclusive:

1. **The shearing sheet.** The Bridges bounce law calls `pow`. One collision in step 78 came out 1 ULP different, and from there the trajectories diverged. Rewriting the identical formula as `exp(-0.234*log(x))` on *both* sides — using functions we had proven bit-identical — made the whole 400-step run match exactly. That control experiment isolates `pow` as the sole cause.
2. **The BS integrator.** BS chooses its own step size using `pow`. Running the three-body test:
 - steps 1 - 2,559: **everything bit-identical**;
 - step 2,560: **all particle positions and velocities still bit-identical**, but the *proposed next step size* differed by exactly 1 ULP (3fce7444d03af04e vs 3fce7444d03af04d).

We then evaluated `pow` with exactly the argument shapes that step-size chooser uses — `pow(error/0.65, 1/(2k+1))` for $k = 1..8$ — over 200,000 samples: **56 disagreements, every one exactly 1 ULP**. Same function, same rate, same magnitude. The physics code is identical; only the platform `pow` differs.

Reproduce that second measurement with:

```
cd C:\work\rebound_rust
cargo build --release --example bs_pow_diff
cd porttest
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
" >nul && cl /nologo /Ox /fp:precise bs_pow_diff.c /Fe:bs_pow_diff.exe'
.\bs_pow_diff.exe
..\target\release\examples\bs_pow_diff.exe
```

What this means for you. Everything REBOUND computes per timestep is bit-exact. If your simulation calls `pow` at run time — in your own force function, in `reb_random_powerlaw`, or through BS's step-size chooser — then on roughly 0.03% of calls the last bit may differ from the C build, and in a chaotic system the two runs will eventually decorrelate. Both remain equally valid; neither is "wrong".

17. Lint policy: why we waive some compiler suggestions

Rust has an optional extra-strict style advisor called **clippy**. Left unconfigured, it reported 469 suggestions across 23 categories on this code.

Every one of those 469 fires on a pattern that deliberately mirrors the C source, and applying clippy's suggestion would either change floating-point evaluation order (which changes results, because floating-point addition is not associative) or destroy the line-by-line correspondence to the C that makes this port reviewable at all.

So rather than either applying them or ignoring them, each category is **waived explicitly in the source**, on one line, with the reason written beside it. You will find the block at the top of `rebound_rust/src/lib.rs` and `reboundx_rust/src/lib.rs`, and repeated in each test and example file (in Rust, a test file is a separate crate and does not inherit the main crate's settings).

Clippy lint	Occurrences	Why it is waived
<code>excessive_precision</code>	274	Physical constants and Solar System data carry the C's exact digits. Truncating them would change values.
<code>identity_op</code>	56	<code>m[0 + 4*0]</code> and <code>y[i*6 + 0]</code> keep the C's row/column and stride arithmetic visible.
<code>erasing_op</code>	49	Same reason: <code>0 * n</code> appears inside index expressions that mirror the C.
<code>needless_range_loop</code>	270	<code>for i in 0..N { ... particles[i] ... }</code> mirrors the C's <code>for(i=0;i<N;i++)</code> . Iterator rewrites obscure aliasing and index relationships.
<code>assign_op_pattern</code>	16	<code>a = a + b</code> mirrors the C statement exactly.
<code>field_reassign_with_default</code>	8	Mirrors C's <code>struct X x = {0}; x.a = ...</code> initialisation idiom.
<code>too_many_arguments</code>	5	C signatures are preserved verbatim, as required.
<code>neg_cmp_op_on_partial_ord</code>	1	Load-bearing — see the warning below.
others (15 kinds)	36	Same category: faithful-but-unidiomatic transcription.

The one waiver you must never "clean up"

`neg_cmp_op_on_partial_ord` fires on two lines that look like they could be tidied, and tidying either of them would reintroduce a bug that made the program hang forever:

```
if !(fastabs(X_max - X_min) > fastabs((X_max + X_min) * 1e-15)) {
    break;
}
```

Clippy suggests rewriting `!(a > b)` as `a <= b`. **Those are not the same thing**. If either value is NaN — "not a number", which really does occur here for nearly-straight-line hyperbolic orbits — then `a > b` is false *and* `a <= b` is false. The C loop is `while (a > b)`, so with a NaN it exits immediately; the "tidied" Rust would never break and would spin forever. This is Defect 1 in section 15.10, and the comment in `lib.rs` says so at the point of the waiver.

The practical consequence of doing it this way:

```
cd C:\work\rebound_rust
cargo build --release          # zero warnings
```

```
cargo clippy --release --all-targets    # zero warnings
```

```
cd C:\work\reboundx_rust
cargo build --release                # zero warnings
cargo clippy --release --all-targets # zero warnings
```

Both crates are clean under both tools, and nothing is hidden: the waivers are in the source where a reviewer reads the code, not buried in a configuration file. To see the underlying suggestions again, delete the `#![allow(clippy:...)]` block from `src/lib.rs` and re-run clippy.

18. Known limitations

1. **pow** — the one maths difference, fully characterised in section 16.
2. **WHFast512 does not integrate on Windows** — in *C* or Rust. Both produce the identical "AVX512 is not supported on your platform." error, because the fast core is GCC/Clang-only assembly that the MSVC build does not compile either.
3. **Excluded subsystems**: the OpenGL 3-D display (`display.c`, `glad.c`) and MPI (`communication_mpi.c`). Neither is part of the Windows C build. The browser-based viewer, which *is* the Windows visualisation path, is ported.
4. **Not carried, and why**: - `reb_simulation_output_screenshot` — needs the browser display round-trip; - `reb_integrator_register` (registering your own integrator at run time) — the Rust integrator set is a closed `enum`; - the WHFast512 AVX-512 assembly core.
5. **The C's own documented restrictions carry over unchanged** — for example MERCURIUS and TRACE emit the same warnings about variational equations and collision-search modes.
6. **cargo clippy is not clean** by design; see section 17.

19. How to reproduce every result yourself

In order, from a fresh machine:

```
# 1. Get the source
cd C:\work
git clone https://github.com/hannorein/rebound.git rebound\rebound
git clone https://github.com/dtamayo/reboundx.git reboundx

# 2. Build the C REBOUND reference
cd rebound\rebound\examples\shearing_sheet
cmd /c 'set PATH=C:\Program Files (x86)\GnuWin32\bin;%PATH% && "C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" && make'
cd ../../src
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && lib /nologo /OUT:librebound_static.lib *.obj'

# 3. Build the C REBOUNDx reference (see section 10b for the VLA shims)
cd ../../..\reboundx\src
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat" >nul && cl /nologo /c /I"../../rebound\rebound\src" /I"." /D_GNU_SOURCE /D_CRT_SECURE_NO_WARNINGS /D_CRT_NONSTDC_NO_WARNINGS /DLIBREBOUNDX /Ox /fp:precise *.c'
```



```
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
" >nul && lib /nologo /OUT:libreboundx.lib *.obj'

# 4. Build the Rust crates and all examples (section 8)
cd ..\..\rebound_rust
cargo build --release
cargo test --release
cd ..\reboundx_rust
cargo build --release
cargo test --release
```

Step 5 — build the C comparison harnesses

A "harness" is a small C program that builds one specific experiment and prints every resulting number as raw bits, so it can be compared with the Rust twin of the same name. There are eleven for REBOUND and five for REBOUNDx.

First open a Visual Studio command prompt, so that `cl` and `lib` are on the path (do this once per terminal window):

```
cmd /c '"C:\Program Files (x86)\Microsoft Visual Studio\18\BuildTools\VC\Auxiliary\Build\vcvars64.bat
"'
```

Build the eleven REBOUND harnesses. They link against `librebound.lib`, which is the small "import library" that goes with `librebound.dll`; copy both of those from `rebound\rebound\src\` into `porttest\` first, because the DLL must sit beside the finished `.exe` files for them to start:

```
cd C:\work\rebound_rust\porttest
copy ..\..\rebound\rebound\src\librebound.lib .
copy ..\..\rebound\rebound\src\librebound.dll .
for %f in (addfmt_test archive_test bs_pow_diff derivatives_test frequency_test integrators_test
  kepler_rectilinear_c libm_diff movetocom_var_c movetocom_var_test problem_test) do cl /nologo /
  OX /fp:precise /I..\..\rebound\rebound\src %f.c librebound.lib /Fe:%f.exe
```

That prints a page of compiler chatter and produces eleven `.exe` files. If you instead see `unresolved external symbol __imp_reb_...`, you linked against `librebound_static.lib`: REBOUND's public functions are declared for DLL import, so the import library is the one to use here.

The REBOUNDx harnesses need two extra object files. REBOUNDx's `gr_full.c` and `interpolation.c` use a C99 feature called a "variable-length array" that Microsoft's compiler does not support, so `porttest\msvc_shim\` holds copies that allocate the same arrays on the heap instead. The arithmetic is unchanged — only where the memory comes from. Build the shims, then the harnesses:

```
cd C:\work\reboundx_rust\porttest\msvc_shim
cl /nologo /c /Ox /fp:precise /I..\..\rebound\rebound\src /I..\..\reboundx\src /
  D_CRT_SECURE_NO_WARNINGS /DLIBREBOUNDX gr_full.c interpolation.c
```

```
cd C:\work\reboundx_rust\porttest
for %f in (tides_spin_pseudo_c tides_spin_kozai_c tides_spin_migration_c rebx_binary_roundtrip_c
  rebx_binary_read_c) do cl /nologo /Ox /fp:precise /I..\..\rebound\rebound\src /I..\..\reboundx\
  src %f.c librebound_static.lib libreboundx.lib msvc_shim\gr_full.obj msvc_shim\interpolation.obj
  /Fe:%f.exe
```

The linker prints LNK4217 and LNK4286 warnings about symbols being both imported and statically

defined. They are expected and harmless: the REBOUNDx library was compiled expecting REBOUND in a DLL, and here it is being given the static library instead. The resulting programs are correct.

Step 6 — run each matched pair and compare

Each harness has a Rust twin with the same name under `examples\`. Run both, then compare. The comparison always works the same way: `Compare-Object` printing nothing means the files are identical.

```
cd C:\work\rebound_rust\porttest
.\integrators_test.exe
..\target\release\examples\integrators_test.exe
Compare-Object (Get-Content ref_c_ias15.txt) (Get-Content ref_rust_ias15.txt)
```

The shearing sheet is the largest single check. Note that the Rust example to run is `shearing_sheet_test`, not `shearing_sheet`: the stock REBOUND example integrates forever by design, so the `_test` variant is the same simulation with a stopping time added.

```
cd C:\work\rebound_rust\porttest
..\target\release\examples\shearing_sheet_test.exe
Get-FileHash state_c_final.txt, state_rust_final.txt -Algorithm SHA256 | Format-List Path, Hash
```

Both hashes must read 75BDAAB7109F125192F56AEB0CCDCAC554AF88A165FE11075EC5A871178521F0.

The three REBOUNDx tidal-spin tests take a stopping time as their argument. Send the C program's error stream to `$null`, because REBOUNDx prints a warning on *every* timestep and that alone produces over a gigabyte of text:

```
cd C:\work\reboundx_rust\porttest
foreach ($t in @(('@('pseudo','62.83185307179586'),@('kozai','1000.0'),@('migration',
'62.83185307179586')))) {
    $n = $t[0]
    & ".\tides_spin_${n}_c.exe" $t[1] 2>$null | Out-Null
    & "..\target\release\examples\tides_spin_${n}.exe" $t[1] 2>$null | Out-Null
    $d = Compare-Object (Get-Content "state_${n}_c.txt") (Get-Content "state_${n}_rust.txt")
    "$n : $(if ($d) { 'MISMATCH' } else { 'BIT-IDENTICAL' })"
}
```

That must print BIT-IDENTICAL three times. The binary-file checks of §15.9 are run with the commands given in that section.

Every command in this document was actually run on the machine described in section 9, and the outputs quoted are the outputs it produced.

20. Credit, citation and licensing

The originals

REBOUND is © Hanno Rein, Shangfei Liu, Dan Tamayo, David S. Spiegel, Tiger Lu, Pejvak Javaheri, Rishit Dagli, Dave O'Hallaron, Ernst Hairer and the REBOUND contributors.

REBOUNDx is © Dan Tamayo, Hanno Rein and the REBOUNDx contributors.

Both are GPL-3.0-or-later. These translations are derivative works under the same license. Every module header names the C file it translates and its copyright holders.

How to cite

From the REBOUND project:

If you use this code or parts of this code for results presented in a scientific publication, we would greatly appreciate a citation. The simplest way to find the citations relevant to the specific setup of your REBOUND simulation is: “`python sim = rebound.Simulation()`
-your setup- `sim.cite()`”

`sim.cite()` is part of the upstream **Python** package (`pip install rebound`), which this Rust port does not reimplement. Use the per-module citation tables in `README.md` instead, which map each feature to the papers `sim.cite()` would name.

At minimum:

- REBOUND — **Rein & Liu 2012**, *Astronomy & Astrophysics* **537**, A128.
- REBOUNDx — **Tamayo, Rein, Shi & Hernandez 2019**, *MNRAS* **491**, 2885 (arXiv:1908.05634).

Plus the paper for the integrator and for each REBOUNDx effect you switch on.

A note on the upstream projects’ AI policy

REBOUND’s README states:

REBOUND is a labour of love, created by people. Please refrain from submitting issues or pull requests that have been generated by an LLM or other fully-automated tools. [...] You may of course use AI assistants for your own work with REBOUND. Just don’t submit any AI generated code.

This translation was produced with AI assistance. That is **exactly the "your own work" case the policy permits**: it is a separate derivative work living in its own repository. Accordingly **nothing from this port has been or will be submitted upstream** — no issues, no pull requests. If you find a problem here, report it here, and please verify against the original C before raising anything with the upstream maintainers, who did not write this and should not be asked to support it.

Versions translated

Project	Version	Commit
REBOUND	5.1.1	<code>dad5f97806ecbb408dcaff728851c64e67f9f6eb</code>
REBOUNDx	5.1.0	latest at time of porting